

▼ 로지스틱 회귀 (Logistic Regression)

- 로지스틱 회귀는 이름에 회귀라는 단어가 들어가지만, 가능한 클래스가 2개인 이진 분류를 위한 모델
- 로지스틱 회귀의 예측 함수 정의

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$
$$\hat{y} = \sigma(w_0 + w_1x_1 + \dots + w_px_p)$$

- σ : 시그모이드 함수
- 로지스틱 회귀 모델은 선형 회귀 모델에 시그모이드 함수를 적용
- 로지스틱 회귀의 학습 목표는 다음과 같은 목적 함수를 최소화 하는 파라미터 w 를 찾는 것

$$BinaryCrossEntropy = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

▼ 로지스틱 회귀 예제

```
import numpy as np
import matplotlib.pyplot as plt
```

```
plt.style.use(['seaborn-whitegrid'])
```

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
```

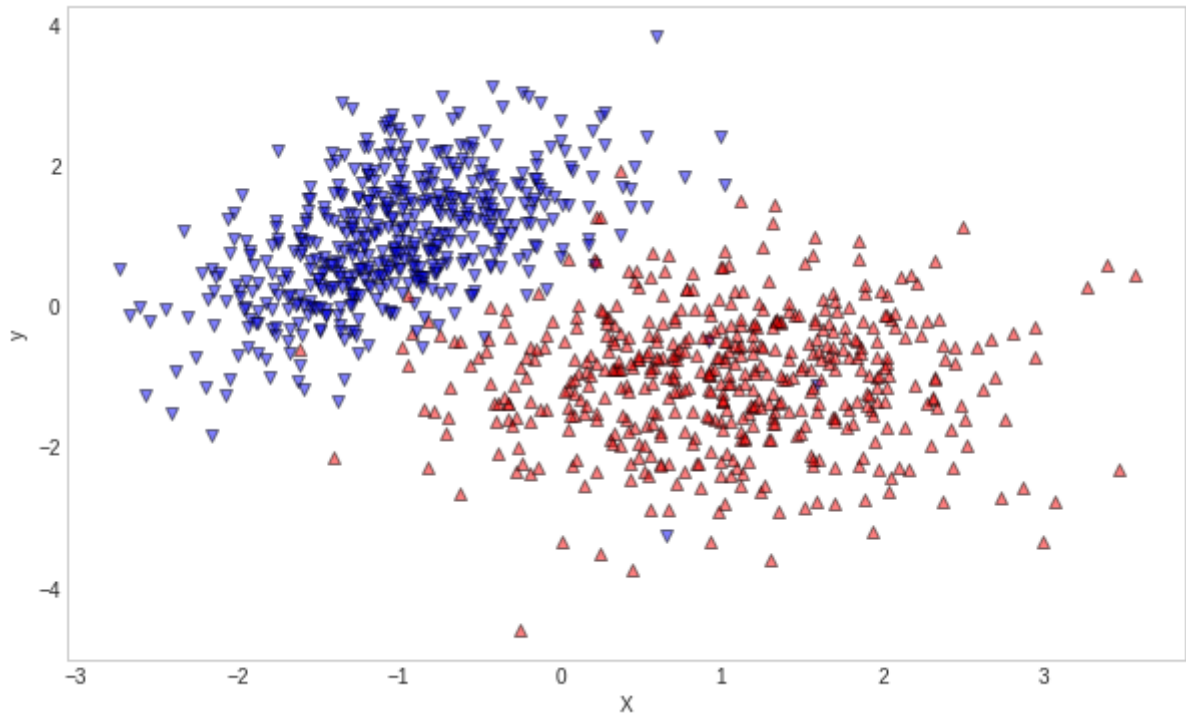
```
samples = 1000
X, y = make_classification(n_samples=samples, n_features=2,
                           n_informative=2, n_redundant=0,
                           n_clusters_per_class=1)
```

```
fig, ax = plt.subplots(1, 1, figsize=(10, 6))
```

```
ax.grid()
ax.set_xlabel('X')
ax.set_ylabel('y')
```

```
for i in range(samples):
    if y[i] == 0:
        ax.scatter(X[i, 0], X[i, 1], edgecolors='k', alpha=0.5, marker='^', color='r')
    else:
        ax.scatter(X[i, 0], X[i, 1], edgecolors='k', alpha=0.5, marker='v', color='b')
```

```
plt.show()
```



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,  
                    intercept_scaling=1, l1_ratio=None, max_iter=100,  
                    multi_class='auto', n_jobs=None, penalty='l2',  
                    random_state=None, solver='lbfgs', tol=0.0001, verbose=0,  
                    warm_start=False)
```

```
print("학습 데이터 점수: {}".format(model.score(X_train, y_train)))  
print("평가 데이터 점수: {}".format(model.score(X_test, y_test)))
```

```
학습 데이터 점수: 0.9825  
평가 데이터 점수: 0.965
```

```
scores = cross_val_score(model, X, y, scoring='accuracy', cv=10)  
print("CV 평균 점수: {}".format(scores.mean()))
```

```
CV 평균 점수: 0.9789999999999999
```

```
model.intercept_, model.coef_
```

```
(array([-0.88979372]), array([[ -3.15674781,  2.41962719]]))
```

```
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5  
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5  
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.02))
```

```

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])

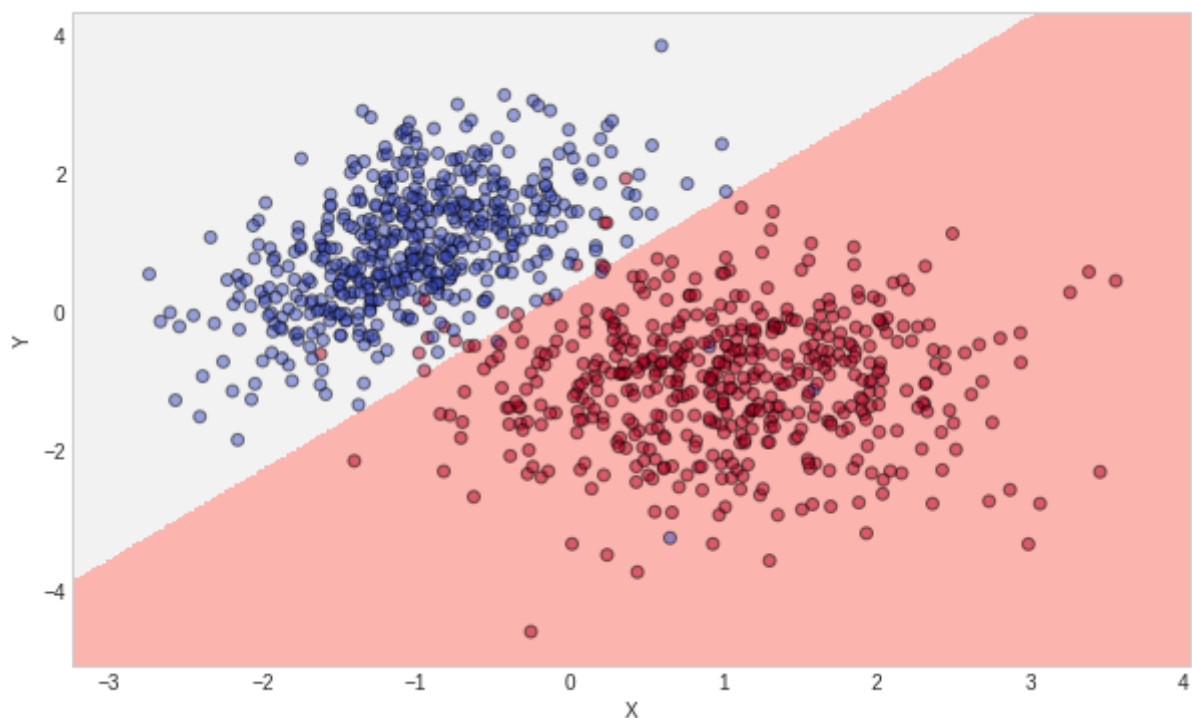
Z = Z.reshape(xx.shape)
plt.figure(1, figsize=(10, 6))
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Pastel1)

plt.scatter(X[:, 0], X[:, 1], c=np.abs(y - 1), edgecolors='k', alpha=0.5, cmap=plt.cm.coolwarm)
plt.xlabel('X')
plt.ylabel('Y')

plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xticks()
plt.yticks()

plt.show()

```



▼ 붓꽃 데이터

```

from sklearn.datasets import load_iris

iris = load_iris()
print(iris.keys())
print(iris.DESCR)

dict_keys(['data', 'target', 'target_names', 'DESCR', 'feature_names', 'filename'])
.. _iris_dataset:

Iris plants dataset
-----

**Data Set Characteristics:**

```

```
:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
  - sepal length in cm
  - sepal width in cm
  - petal length in cm
  - petal width in cm
  - class:
    - Iris-Setosa
    - Iris-Versicolour
    - Iris-Virginica
```

```
:Summary Statistics:
```

	Min	Max	Mean	SD	Class Correlation
sepal length:	4.3	7.9	5.84	0.83	0.7826
sepal width:	2.0	4.4	3.05	0.43	-0.4194
petal length:	1.0	6.9	3.76	1.76	0.9490 (high!)
petal width:	0.1	2.5	1.20	0.76	0.9565 (high!)

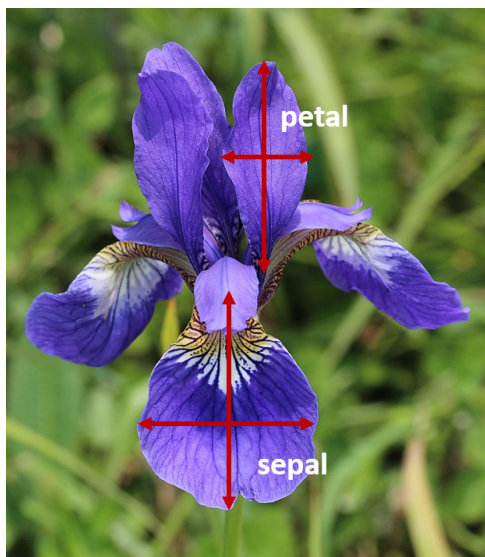
```
:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988
```

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken from Fisher's paper. Note that it's the same as in R, but not as in the UCI Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the pattern recognition literature. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The data set contains 3 classes of 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

.. topic:: References

- Fisher, R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarthy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.



Iris virginica



Iris versicolor



Iris setosa

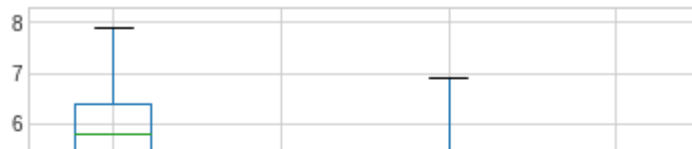
```
import pandas as pd
```

```
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
species = pd.Series(iris.target, dtype='category')
species = species.cat.rename_categories(iris.target_names)
iris_df['species'] = species
```

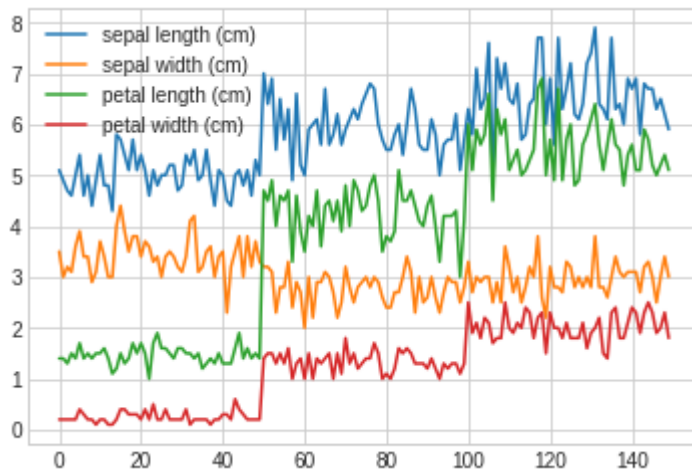
```
iris_df.describe()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
iris_df.boxplot();
```



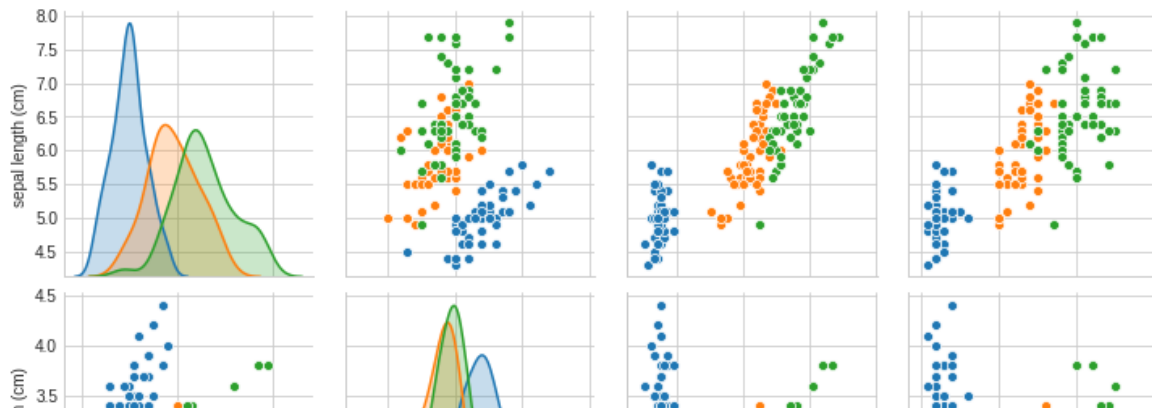
```
iris_df.plot();
```



```
import seaborn as sns
```

```
sns.pairplot(iris_df, hue='species');
```

```
/usr/local/lib/python3.6/dist-packages/statsmodels/tools/_testing.py:19: FutureWarning
import pandas.util.testing as tm
```



▼ 붓꽃 데이터에 대한 로지스틱 회귀

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(iris.data[:, [2, 3]], iris.target,
                                                    test_size=0.2, random_state=1, stratify=iris.ta
```

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(solver='lbfgs', multi_class='auto', C=100.0, random_state=1)
model.fit(X_train, y_train)
```

```
LogisticRegression(C=100.0, class_weight=None, dual=False, fit_intercept=True,
                    intercept_scaling=1, l1_ratio=None, max_iter=100,
                    multi_class='auto', n_jobs=None, penalty='l2',
                    random_state=1, solver='lbfgs', tol=0.0001, verbose=0,
                    warm_start=False)
```

```
print("학습 데이터 점수: {}".format(model.score(X_train, y_train)))
print("평가 데이터 점수: {}".format(model.score(X_test, y_test)))
```

```
학습 데이터 점수: 0.9666666666666667
평가 데이터 점수: 0.9666666666666667
```

```
import numpy as np
X = np.vstack((X_train, X_test))
y = np.hstack((y_train, y_test))
```

```
from matplotlib.colors import ListedColormap
```

```
x1_min, x1_max = X[:, 0].min()-1, X[:, 0].max()+1
x2_min, x2_max = X[:, 1].min()-1, X[:, 1].max()+1
xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, 0.02),
                        np.arange(x2_min, x2_max, 0.02))
Z = model.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
Z = Z.reshape(xx1.shape)
```

```
species = ('Setosa', 'Versicolour', 'Virginica')
```



```

n_jobs=None, penalty='l2',
random_state=None, solver='lbfgs',
tol=0.0001, verbose=0,
warm_start=False),
iid='deprecated', n_jobs=2,
param_grid=[{'C': [2.0, 2.2, 2.4, 2.6, 2.8],
              'penalty': ['l1', 'l2']}],
pre_dispatch='2*n_jobs', refit=True, return_train_score=False,
scoring='accuracy', verbose=0)

```

```
result = gs.fit(iris.data, iris.target)
```

/usr/local/lib/python3.6/dist-packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

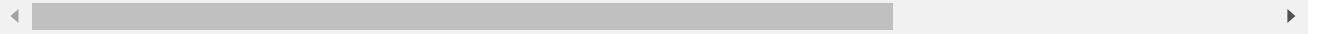
Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)



```

print(gs.best_estimator_)
print("최적 점수: {}".format(gs.best_score_))
print("최적 파라미터: {}".format(gs.best_params_))
pd.DataFrame(result.cv_results_)

```

```
LogisticRegression(C=2.4, class_weight=None, dual=False, fit_intercept=True,
                    intercept_scaling=1, l1_ratio=None, max_iter=100,
                    multi_class='auto', n_jobs=None, penalty='l2',
                    random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                    warm_start=False)
```

최적 점수: 0.9800000000000001

최적 파라미터: {'C': 2.4, 'penalty': 'l2'}

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_pena
0	0.000234	0.000388	0.000000	0.000000	2	
1	0.050777	0.002902	0.000574	0.000025	2	

▼ 유방암 데이터

```
from sklearn.datasets import load_breast_cancer
```

```
cancer = load_breast_cancer()
print(cancer.keys())
print(cancer.DESCR)
```

```
dict_keys(['data', 'target', 'target_names', 'DESCR', 'feature_names', 'filename'])
.. _breast_cancer_dataset:
```

Breast cancer wisconsin (diagnostic) dataset

****Data Set Characteristics:****

:Number of Instances: 569

:Number of Attributes: 30 numeric, predictive attributes and the class

:Attribute Information:

- radius (mean of distances from center to points on the perimeter)
- texture (standard deviation of gray-scale values)
- perimeter
- area
- smoothness (local variation in radius lengths)
- compactness (perimeter² / area - 1.0)
- concavity (severity of concave portions of the contour)
- concave points (number of concave portions of the contour)
- symmetry
- fractal dimension ("coastline approximation" - 1)

The mean, standard error, and "worst" or largest (mean of the three largest values) of these features were computed for each image, resulting in 30 features. For instance, field 3 is Mean Radius, field 13 is Radius SE, field 23 is Worst Radius.

- class:
 - WDBC-Malignant

:Summary Statistics:

	Min	Max
radius (mean):	6.981	28.11
texture (mean):	9.71	39.28
perimeter (mean):	43.79	188.5
area (mean):	143.5	2501.0
smoothness (mean):	0.053	0.163
compactness (mean):	0.019	0.345
concavity (mean):	0.0	0.427
concave points (mean):	0.0	0.201
symmetry (mean):	0.106	0.304
fractal dimension (mean):	0.05	0.097
radius (standard error):	0.112	2.873
texture (standard error):	0.36	4.885
perimeter (standard error):	0.757	21.98
area (standard error):	6.802	542.2
smoothness (standard error):	0.002	0.031
compactness (standard error):	0.002	0.135
concavity (standard error):	0.0	0.396
concave points (standard error):	0.0	0.053
symmetry (standard error):	0.008	0.079
fractal dimension (standard error):	0.001	0.03
radius (worst):	7.93	36.04

```
import pandas as pd
```

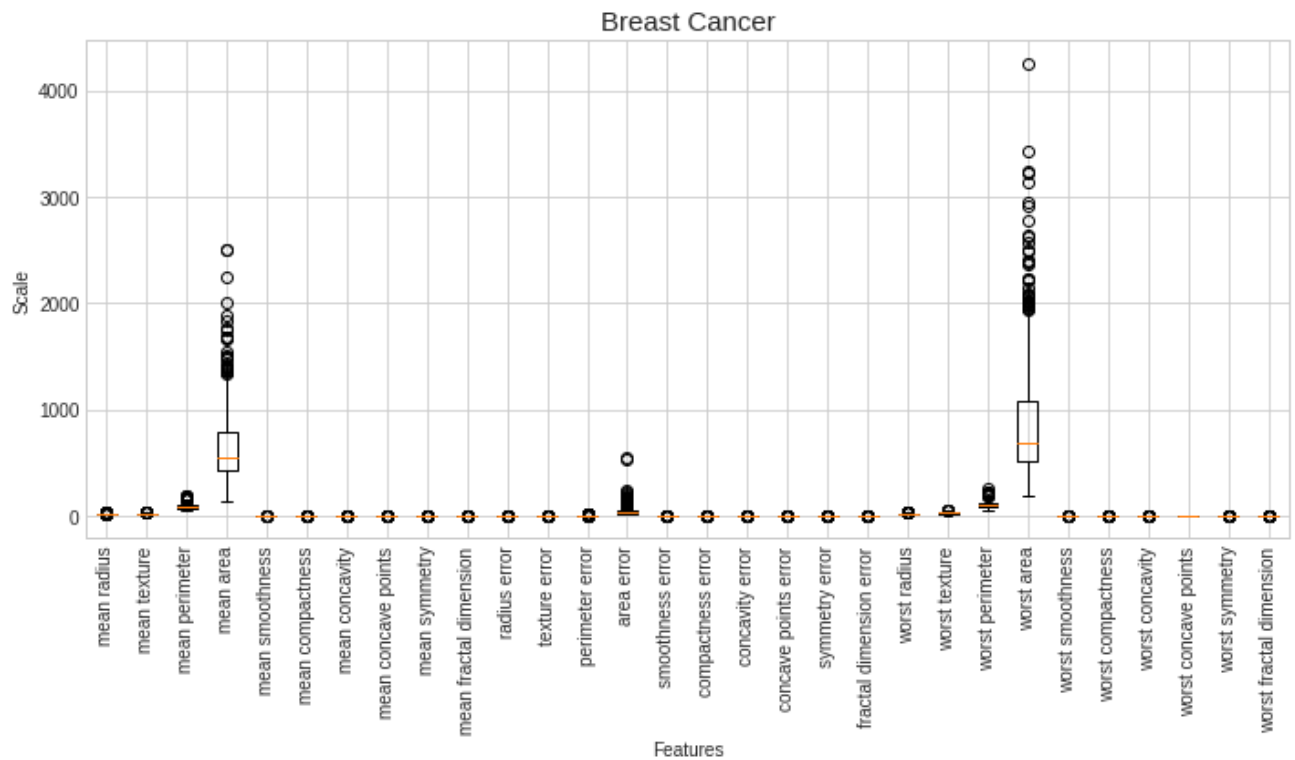
```
cancer_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
cancer_df['Target'] = cancer.target
cancer_df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	s
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	

```
cancer_df.describe()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concave points
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.084459
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.076229
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.020000

```
fig = plt.figure(figsize=[10, 6])
plt.title('Breast Cancer', fontsize=15)
plt.boxplot(cancer.data)
plt.xticks(np.arange(30)+1, cancer.feature_names, rotation=90)
plt.xlabel('Features')
plt.ylabel('Scale')
plt.tight_layout()
```



▼ 유방암 데이터에 대한 로지스틱 회귀

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

```
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
model = LogisticRegression(max_iter=3000)
model.fit(X_train, y_train)
```

```
LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                    intercept_scaling=1, l1_ratio=None, max_iter=3000,
                    multi_class='auto', n_jobs=None, penalty='l2',
                    random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                    warm_start=False)
```

```
print("학습 데이터 점수: {}".format(model.score(X_train, y_train)))
print("평가 데이터 점수: {}".format(model.score(X_test, y_test)))
```

학습 데이터 점수: 0.9553990610328639

평가 데이터 점수: 0.9300699300699301

▼ 확률적 경사 하강법 (Stochastic Gradient Descent)

- 모델을 학습 시키기 위한 간단한 방법
- 학습 파라미터에 대한 손실 함수의 기울기를 구해 기울기가 최소화 되는 방향으로 학습

$$\frac{\partial L}{\partial w} = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$
$$w' = w - \alpha \frac{\partial L}{\partial w}$$

- scikit-learn에서는 선형 SGD 회귀와 SGD 분류를 지원

▼ SGD를 사용한 선형 회귀 분석

```
from sklearn.linear_model import SGDRegressor
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

```
X, y = load_boston(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
model = make_pipeline(StandardScaler(), SGDRegressor(loss='squared_loss'))
model.fit(X_train, y_train)
```

```
Pipeline(memory=None,
          steps=[('standardscaler',
                  StandardScaler(copy=True, with_mean=True, with_std=True)),
                 ('sgdregressor',
                  SGDRegressor(alpha=0.0001, average=False, early_stopping=False,
                               epsilon=0.1, eta0=0.01, fit_intercept=True,
```

```

        l1_ratio=0.15, learning_rate='invscaling',
        loss='squared_loss', max_iter=1000,
        n_iter_no_change=5, penalty='l2', power_t=0.25,
        random_state=None, shuffle=True, tol=0.001,
        validation_fraction=0.1, verbose=0,
        warm_start=False))],
        verbose=False)

```

```

print("학습 데이터 점수: {}".format(model.score(X_train, y_train)))
print("평가 데이터 점수: {}".format(model.score(X_test, y_test)))

```

```

학습 데이터 점수: 0.7201986493736362
평가 데이터 점수: 0.7871909593460191

```

▼ 붓꽃 데이터에 대한 SGD 분류

```

from sklearn.linear_model import SGDClassifier
from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

```

```

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

```

```

model = make_pipeline(StandardScaler(), SGDClassifier(loss='log'))
model.fit(X_train, y_train)

```

```

Pipeline(memory=None,
       steps=[('standardscaler',
               StandardScaler(copy=True, with_mean=True, with_std=True)),
              ('sgdclassifier',
               SGDClassifier(alpha=0.0001, average=False, class_weight=None,
                             early_stopping=False, epsilon=0.1, eta0=0.0,
                             fit_intercept=True, l1_ratio=0.15,
                             learning_rate='optimal', loss='log',
                             max_iter=1000, n_iter_no_change=5, n_jobs=None,
                             penalty='l2', power_t=0.5, random_state=None,
                             shuffle=True, tol=0.001, validation_fraction=0.1,
                             verbose=0, warm_start=False))],
       verbose=False)

```

```

print("학습 데이터 점수: {}".format(model.score(X_train, y_train)))
print("평가 데이터 점수: {}".format(model.score(X_test, y_test)))

```

```

학습 데이터 점수: 0.9553571428571429
평가 데이터 점수: 0.9473684210526315

```

▼ 유방암 데이터에 대한 SGD 분류

```
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
model = make_pipeline(StandardScaler(), SGDClassifier(loss='log'))
model.fit(X_train, y_train)
```

```
Pipeline(memory=None,
       steps=[('standardscaler',
               StandardScaler(copy=True, with_mean=True, with_std=True)),
              ('sgdclassifier',
               SGDClassifier(alpha=0.0001, average=False, class_weight=None,
                             early_stopping=False, epsilon=0.1, eta0=0.0,
                             fit_intercept=True, l1_ratio=0.15,
                             learning_rate='optimal', loss='log',
                             max_iter=1000, n_iter_no_change=5, n_jobs=None,
                             penalty='l2', power_t=0.5, random_state=None,
                             shuffle=True, tol=0.001, validation_fraction=0.1,
                             verbose=0, warm_start=False))],
       verbose=False)
```

```
print("학습 데이터 점수: {}".format(model.score(X_train, y_train)))
print("평가 데이터 점수: {}".format(model.score(X_test, y_test)))
```

```
학습 데이터 점수: 0.9812206572769953
평가 데이터 점수: 0.951048951048951
```